GPU computing

V100, A100, H100 ◂

CUDA ◂

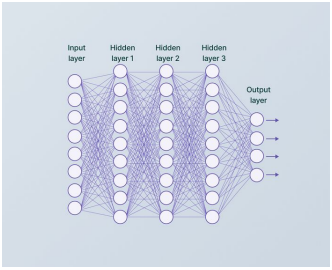
CuDNN ◂

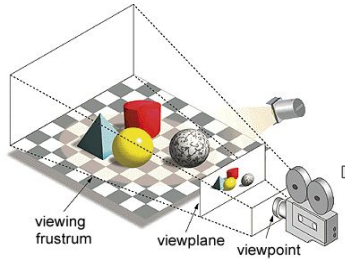
AMP ◂

This part presents GPU accelerators: why they compute much faster than a CPU, how they are organized, and what this means for us as PyTorch users. We will look at the three generations available on Jean Zay (V100, A100, H100), the CUDA and cuDNN ecosystem, then mixed precision. Keep this in mind from now on: the GPU has huge compute power, but a memory much smaller than the RAM of a node. A large part of the optimizations is about managing this trade-off.

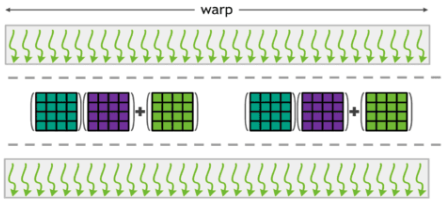
Cette partie présente les accélérateurs GPU : pourquoi ils calculent beaucoup plus vite qu'un CPU, comment ils sont organisés, et ce que ça implique pour nous, utilisateurs de PyTorch. On verra les trois générations présentes sur Jean Zay (V100, A100, H100), l'écosystème CUDA et cuDNN, puis la précision mixte. À garder en tête dès maintenant : le GPU a une puissance de calcul énorme, mais une mémoire bien plus petite que la RAM d'un nœud. Une bonne partie des optimisations consiste à gérer ce compromis.

GPU computing



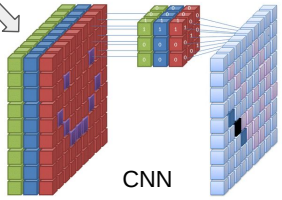


GPU Rendering & Game Graphics Pipeline



Matrix Multiply-accumulate operations

NN



CNN

GPUs were designed for 3D rendering in video games: computing the color of millions of pixels, each one independently, with the same operations.

These are mostly matrix multiplications and additions, executed in parallel by groups of threads called warps.

Deep learning relies on exactly the same kind of computation. A dense layer is a matrix multiplication, and a convolution can be turned into a matrix multiplication.

So the GPU is naturally suited to classic neural networks as well as convolutional networks.

The difference with a CPU is a matter of philosophy: a CPU has a few very powerful cores and a lot of control logic, to run varied tasks quickly. A GPU has thousands of simple cores, built to apply the same operation to a huge amount of data at the same time.

Les GPU ont été conçus pour le rendu 3D des jeux vidéo : calculer la couleur de millions de pixels, chacun indépendamment, avec les mêmes opérations.

Ce sont essentiellement des multiplications de matrices et des additions, exécutées en parallèle par paquets de threads appelés warps.

Il se trouve que l'apprentissage profond repose sur exactement le même type de calcul. Une couche dense est une multiplication de matrices,

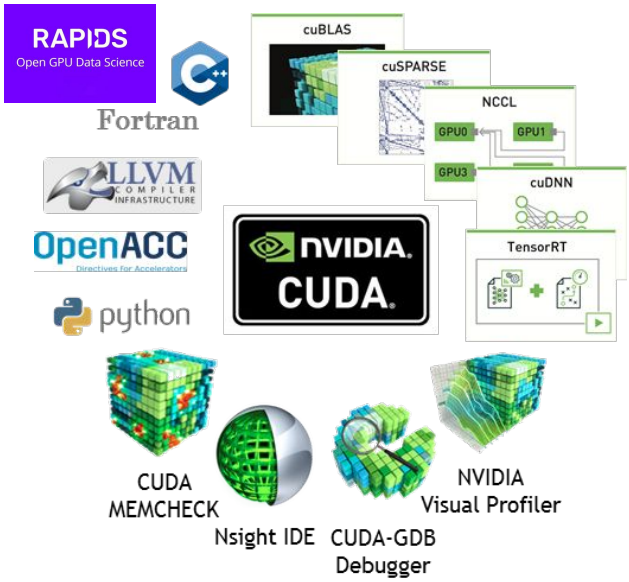
une convolution peut se ramener à une multiplication de matrices.

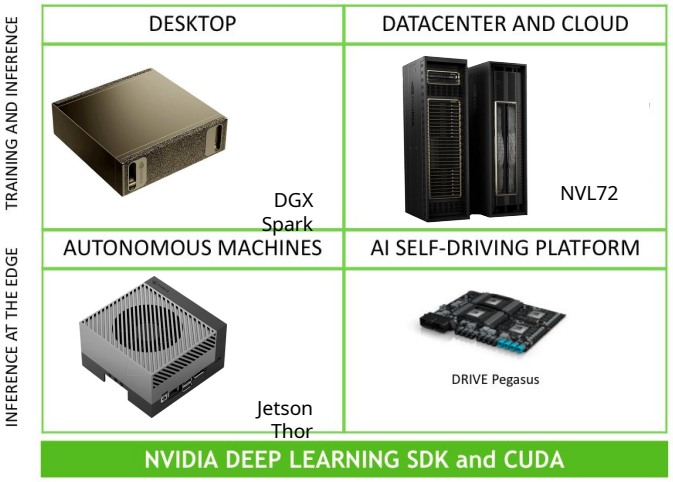
Le GPU est donc naturellement adapté aux réseaux de neurones classiques comme aux réseaux convolutionnels.

La différence avec un CPU tient à la philosophie : un CPU a quelques cœurs très puissants et beaucoup de logique de contrôle, pour exécuter vite des tâches

variées. Un GPU a des milliers de cœurs simples, faits pour appliquer la même opération à énormément de données en même temps.

NVIDIA Galaxy





Source : NVidia

NVIDIA does not only sell chips: its real strength is the software ecosystem built around CUDA, its GPU programming language.

Around CUDA there are optimized libraries that we use without knowing it: cuBLAS for linear algebra, cuDNN for neural networks, NCCL for communication between GPUs, TensorRT for inference, RAPIDS for data science.

On top of that come development tools: compilers, debuggers, and the Nsight profiling tools, which we will use in the profiler part.

On the right, the hardware range goes from the workstation to the supercomputer, down to embedded systems and self-driving cars, all with the same software base. This is what makes moving from one GPU to another fairly transparent for the user.

NVIDIA ne vend pas seulement des puces : son vrai atout est l'écosystème logiciel construit autour de CUDA, son langage de programmation des GPU.

Autour de CUDA gravitent des bibliothèques optimisées que nous utilisons sans le savoir : cuBLAS pour l'algèbre linéaire, cuDNN pour les réseaux de

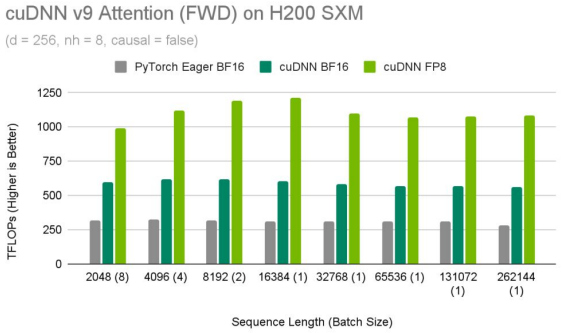
neurones, NCCL pour la communication entre GPU, TensorRT pour l'inférence, RAPIDS pour la data science.

S'y ajoutent des outils de développement : compilateurs, débogueurs, et les outils de profilage Nsight, qu'on utilisera dans la partie profiler.

À droite, la gamme matérielle va du poste de travail au supercalculateur, jusqu'à l'embarqué et la voiture autonome, avec le même socle logiciel. C'est ce qui rend le passage d'un GPU à un autre relativement transparent pour l'utilisateur.

CuDNN





https://developer.nvidia.com/blog/accelerating-transformers-with-nvidia-cudnn-9/

CUDA engineering for deep learning on GPU is handled by cuDNN.

Recommendation: to optimize the use of Tensor Cores and Cuda Cores: Use tensors with dimensions (batch size, sample size, channel, layer dimension, etc.) **multiples of 8**!!

cuDNN is the CUDA library dedicated to neural networks. It provides highly optimized implementations of convolutions, normalizations and activations. PyTorch calls it automatically: when you write a convolution, a cuDNN kernel runs.

Consequence: all the CUDA engineering of the next slides is handled for us. We present it to understand what happens, not to write it.

The chart shows a practical point: on the same hardware, a new cuDNN version can clearly speed up training. Using recent versions of the PyTorch modules matters.

One recommendation to remember: size your tensors in multiples of 8 (batch size, number of channels, number of filters, layer dimensions).

Tensor Cores work on blocks of fixed size; dimensions that are not multiples force padding or a fallback to slower cores.

cuDNN est la bibliothèque CUDA dédiée aux réseaux de neurones. Elle fournit des implémentations très optimisées des convolutions, des normalisations, des
activations. PyTorch l'appelle automatiquement : quand vous écrivez une convolution, c'est un kernel cuDNN qui s'exécute.

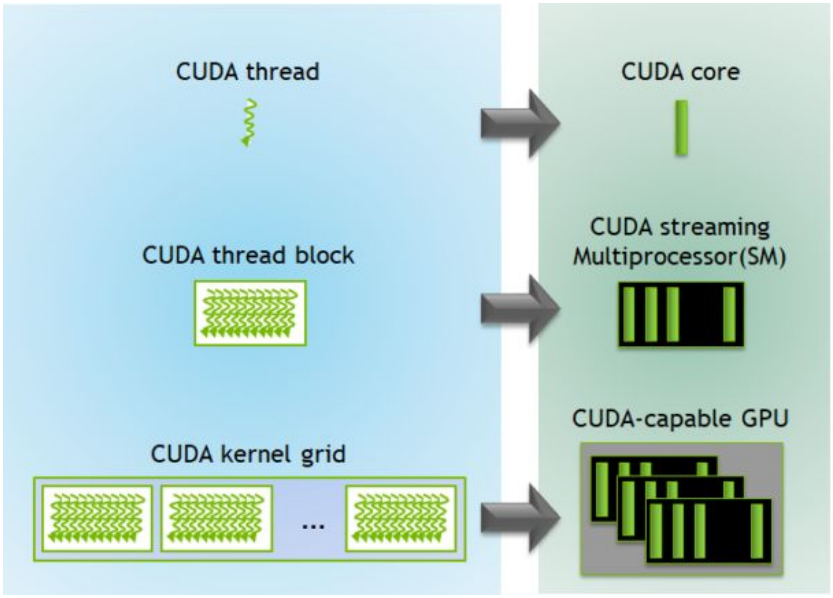
Conséquence : toute l'ingénierie CUDA des slides suivantes est gérée pour nous. On la présente pour comprendre ce qui se passe, pas pour l'écrire.

Le graphique illustre un point pratique : à matériel identique, une nouvelle version de cuDNN peut nettement accélérer un entraînement. Utiliser des versions
récentes des modules PyTorch compte.

Une seule recommandation à retenir : dimensionner les tenseurs en multiples de 8 (taille de batch, nombre de canaux, de filtres, dimensions des couches).

Les Tensor Cores travaillent sur des blocs de taille fixe ; des dimensions non multiples obligent à compléter ou à repasser sur des cœurs plus lents.

GPU computing : CUDA



Source : NVidia

The CUDA vocabulary, which we will find again in the profiler traces.

A kernel is a function executed on the GPU. It is split into threads: each thread handles a small piece of the computation, for example one element of the result. Threads are grouped into blocks, and all the blocks of a kernel form a grid.

On the hardware side, the mapping is: a thread runs on a CUDA core; a whole block is given to one multiprocessor, the SM, and its threads share the fast memory of this SM; the grid is spread over all the SMs of the GPU.

Inside an SM, threads move forward in groups of 32, the warps, which execute the same instruction at the same time. The warp is the basic unit of execution on a GPU.

Le vocabulaire CUDA, qu'on retrouvera dans les traces du profiler.

Un kernel est une fonction exécutée sur le GPU. Il est découpé en threads : chaque thread traite un petit morceau du calcul, par exemple un élément du résultat. Les threads sont

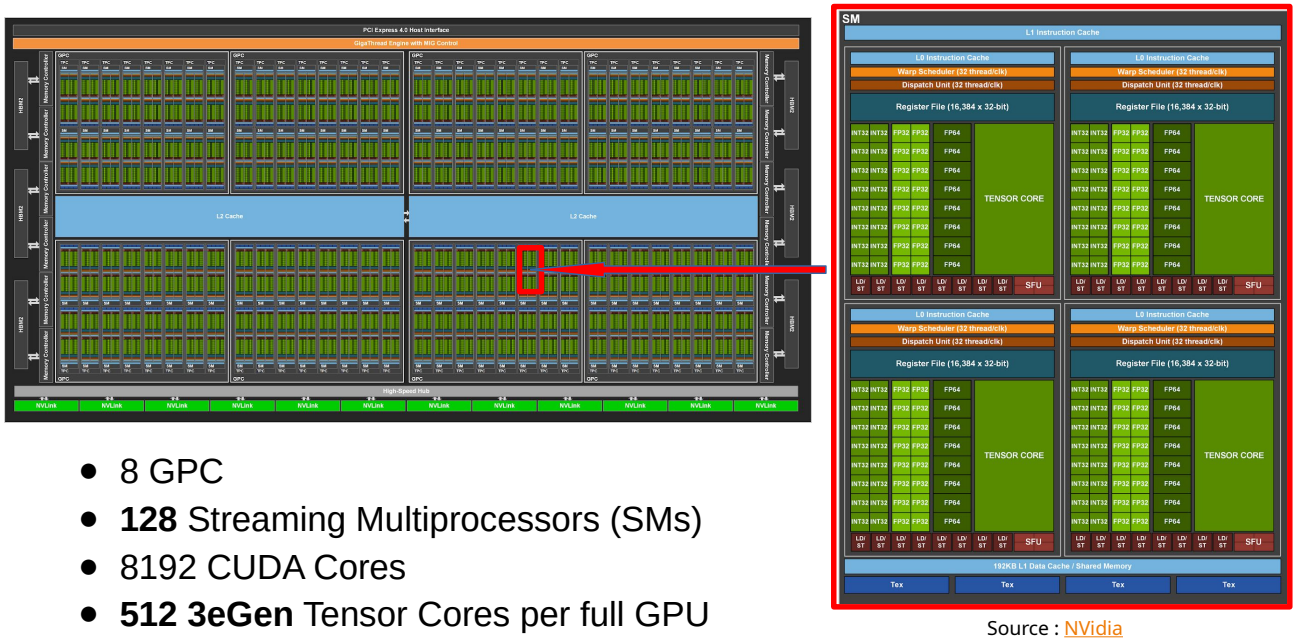
regroupés en blocs, et l'ensemble des blocs d'un kernel forme une grille.

Côté matériel, la correspondance est la suivante : un thread s'exécute sur un cœur CUDA ; un bloc est confié en entier à un multiprocesseur, le SM, et ses threads partagent la

mémoire rapide de ce SM ; la grille se répartit sur tous les SM du GPU.

À l'intérieur d'un SM, les threads avancent par paquets de 32, les warps, qui exécutent la même instruction en même temps. C'est l'unité de base de l'exécution sur GPU.

A100 Architecture



- 8 GPC
- **128** Streaming Multiprocessors (SMs)
- 8192 CUDA Cores
- **512 3eGen** Tensor Cores per full GPU

Source : NVidia

The A100 (Ampere architecture) keeps the same organization, with more resources: 8 GPCs and 128 SMs on the full chip. The product version enables 108 of them, i.e. 6,912 FP32 CUDA cores and 432 Tensor Cores.

Worth noting: the A100 has only 4 Tensor Cores per SM, versus 8 on the V100. But they are third-generation Tensor Cores, each about 4 times more powerful: per SM, the matrix throughput doubles.

Above all, this generation brings new precisions to the Tensor Cores: TF32 and BF16, which we will see later.

On Jean Zay, the A100s have 80 GB of memory, with 8 GPUs per node. This is the GPU used for the labs.

L'A100 (architecture Ampere) reprend la même organisation, avec plus de ressources : 8 GPC et 128 SM sur la puce complète. La version commercialisée en active 108, soit 6 912

cœurs CUDA FP32 et 432 Tensor Cores.

Point notable : l'A100 n'a plus que 4 Tensor Cores par SM, contre 8 sur le V100. Mais ce sont des Tensor Cores de troisième génération, chacun environ 4 fois plus puissant : à SM

égal, la puissance matricielle double.

Cette génération apporte surtout de nouvelles précisions pour les Tensor Cores : TF32 et BF16, qu'on verra plus loin.

Sur Jean Zay, les A100 ont 80 Go de mémoire, à 8 GPU par nœud. C'est le GPU utilisé pour les TP.

H100 Architecture





- 8 GPC
- **132** Streaming Multiprocessors (SMs)
- 16896 CUDA Cores
- **528 4eGen** Tensor Cores per H100 SXM5

Source : NVidia

The H100 (Hopper architecture), the most recent generation on Jean Zay.

The SXM5 version has 132 SMs, 16,896 CUDA cores and 528 fourth-generation Tensor Cores. The full chip goes up to 144 SMs.

Main new features: FP8 precision on the Tensor Cores, used for very large models, and the Transformer Engine, which manages it automatically. In the SM we also see a memory-copy accelerator (Tensor Memory Accelerator) and a larger L1 cache.

Memory moves to HBM3: 80 GB, with a bandwidth of about 3.35 TB/s, versus about 2 TB/s for the A100.

On Jean Zay, the H100s are grouped 4 GPUs per node. The message of these three slides: at each generation, Tensor Core power grows much faster than the power of the classic cores.

Le H100 (architecture Hopper), la génération la plus récente sur Jean Zay.

La version SXM5 compte 132 SM, 16 896 cœurs CUDA et 528 Tensor Cores de quatrième génération. La puce complète va jusqu'à 144 SM.

Nouveautés principales : la précision FP8 sur les Tensor Cores, utilisée pour les très gros modèles, et le Transformer Engine qui la gère automatiquement. On voit aussi dans le SM

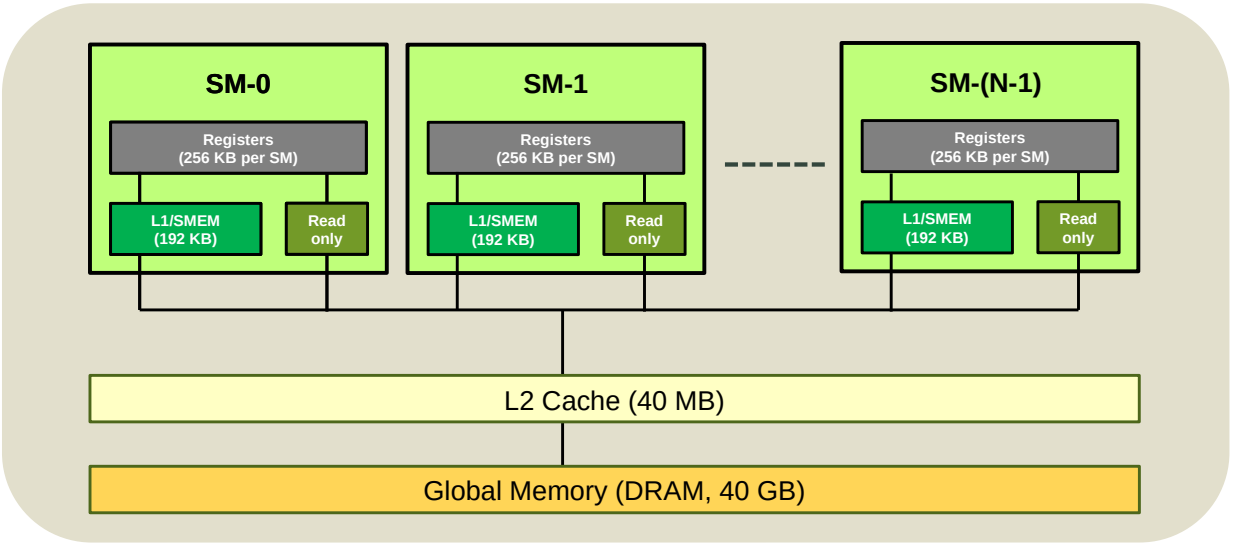
un accélérateur de copies mémoire (Tensor Memory Accelerator) et un cache L1 plus grand.

La mémoire passe en HBM3 : 80 Go, avec une bande passante d'environ 3,35 To/s, contre environ 2 To/s pour l'A100.

Sur Jean Zay, les H100 sont regroupés à 4 GPU par nœud. Le message à retenir de ces trois slides : à chaque génération, la puissance des Tensor Cores progresse bien plus vite

que celle des cœurs classiques.

Optimized memory management



GPU memory is organized like a funnel: the closer to the compute cores, the faster and smaller the memory.

Closest: the registers, 256 KB per SM, private to each thread. Then the L1 cache, which also serves as shared memory between the threads of a block: 192 KB per SM on the A100.

Then the L2 cache, 40 MB, shared by all SMs. Finally the global memory, the HBM: 40 or 80 GB depending on the model, about 2 TB/s.

Why it matters for us: fetching data from global memory costs much more than computing on it.

The best kernels load the data once into fast memory and do as much computation as possible on it before writing it back.

This is the principle of kernel fusion, and of FlashAttention for transformers.

La mémoire du GPU est organisée en entonnoir : plus on est proche des cœurs de calcul, plus la mémoire est rapide et petite.

Au plus près : les registres, 256 Ko par SM, privés à chaque thread. Ensuite le cache L1, qui sert aussi de mémoire partagée entre les threads d'un même bloc : 192 Ko par SM sur
l'A100.

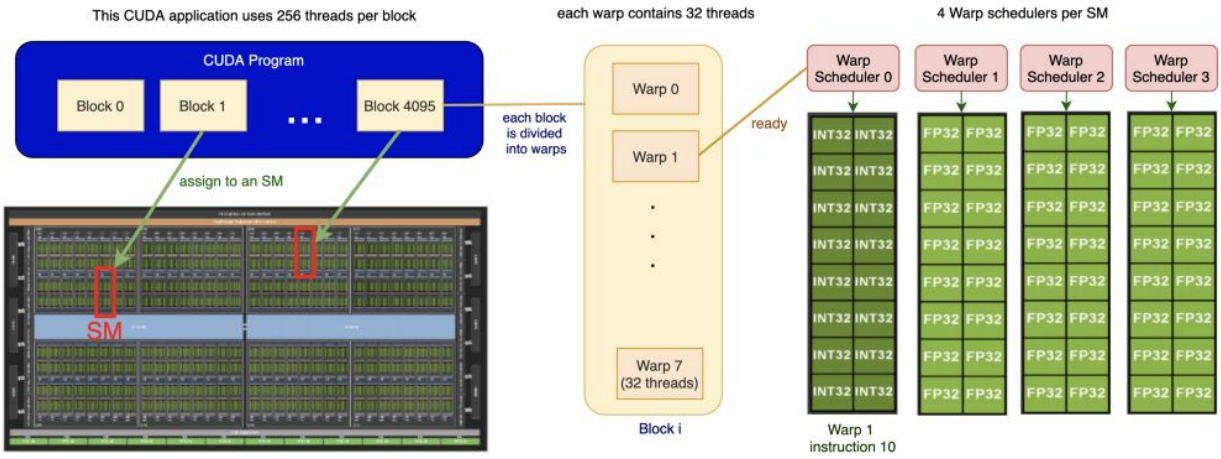
Puis le cache L2, 40 Mo, commun à tous les SM. Enfin la mémoire globale, la HBM : 40 ou 80 Go selon le modèle, environ 2 To/s.

Pourquoi c'est important pour nous : aller chercher une donnée en mémoire globale coûte beaucoup plus cher que la calculer.

Les meilleurs kernels chargent les données une fois dans la mémoire rapide et font le maximum de calcul dessus avant de les réécrire.

C'est le principe de la fusion de kernels, et de FlashAttention pour les transformers.

CUDA Engineering



Optimization :

- Block occupancy
- Spread blockss over all SM

Advanced Optimization :

- Kernel Fusion to override initialization times (fewer launches and memory round trips)

How a CUDA program actually runs. The program launches a grid of blocks; each block is assigned to an SM; inside the SM, the block is split into warps of 32 threads, which the 4 schedulers dispatch to the cores.

Two optimization rules follow from this.

– Occupancy: each SM needs enough warps. When a warp waits for data from memory, the scheduler switches to another one. With too few warps, the cores sit idle.

– Distribution: the blocks must be spread over all the SMs. Like an ice-cube tray: with 109 blocks for 108 SMs, the last block runs alone while the other SMs wait.

The advanced optimization is kernel fusion: grouping several operations into a single kernel avoids successive launches and round trips to global memory.

torch.compile does it automatically. We will measure occupancy with Nsight Compute in the profiler part.

Comment un programme CUDA s'exécute concrètement. Le programme lance une grille de blocs ; chaque bloc est assigné à un SM, dans le SM, le bloc

est découpé en warps de 32 threads, que les 4 ordonnanceurs distribuent aux cœurs.

Deux règles d'optimisation en découlent.

– L'occupancy : il faut assez de warps sur chaque SM. Quand un warp attend une donnée de la mémoire, l'ordonnanceur passe à un autre. Avec trop peu de warps, les cœurs restent inactifs.

– La répartition : les blocs doivent se répartir sur tous les SM. Comme un bac à glaçons : si on a 109 blocs pour 108 SM, le dernier bloc tourne seul pendant que les autres SM attendent.

L'optimisation avancée est la fusion de kernels : regrouper plusieurs opérations dans un seul kernel évite des lancements successifs et des allers-retours

vers la mémoire globale.

torch.compile le fait automatiquement. On mesurera l'occupancy avec Nsight Compute dans la partie profiler.

TP1 : Accélération GPU





- Envoyer le calcul sur le GPU
- Test Mémoire

Tensor Cores

Tensor Cores ◂

Precisions ◂

AMP ◂

Channel last memory format ◂

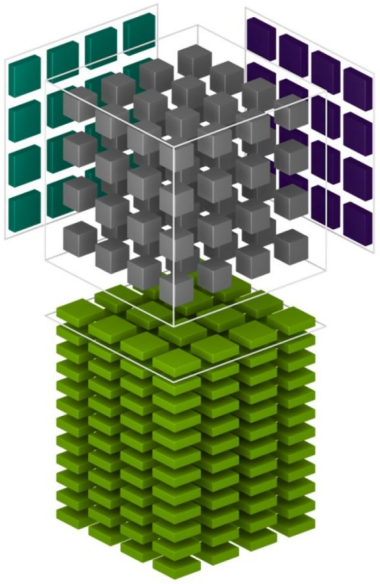
Second part: how to really use the GPU. We will see the Tensor Cores, the units that provide most of the compute power; the numerical precisions they use; automatic mixed precision (AMP), which lets us use them with two lines of code; and the channels-last memory format, which makes them even more efficient for convolutions.

Deuxième partie : comment exploiter vraiment le GPU. On va voir les Tensor Cores, les unités qui fournissent l'essentiel de la puissance de calcul, les

précisions numériques qu'ils utilisent, la précision mixte automatique (AMP) qui permet de s'en servir en deux lignes de code, et le format mémoire

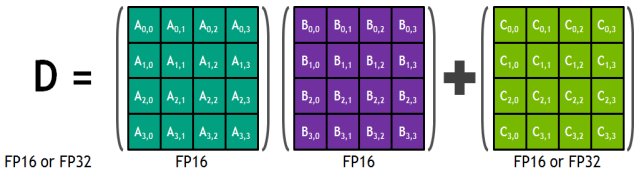
channels last, qui les rend encore plus efficaces pour les convolutions.

Tensor Cores



CUDA Core are specialized for **vector computing**.

Tensor Cores are specialized for **matrix calculation**.



Each **V100 Tensor Core** is capable of processing 64 multiply-adds in 1 clock time.

Source : NVidia

Classic CUDA cores do one operation on one scalar at a time: one multiply-add per cycle. Tensor Cores are specialized in matrix computation: in one cycle, they compute D = A × B + C on small matrices.

On the V100, each Tensor Core handles 4×4 matrices: 64 multiply-adds per cycle. The inputs are FP16, and accumulation can be done in FP32 to keep the precision of the result. Later generations handle larger blocks: 256 multiply-adds per cycle for an A100 Tensor Core.

Since deep learning is 80–90% matrix multiplications and convolutions, Tensor Cores provide most of the GPU's power. But only under two conditions: use a precision they accept, and suitable dimensions. Otherwise, the computation falls back to the classic CUDA cores, which are much slower.

Les cœurs CUDA classiques font une opération sur un scalaire à la fois : une multiplication-addition par cycle. Les Tensor Cores, eux, sont spécialisés dans le calcul matriciel : en un

cycle, ils calculent D = A × B + C sur de petites matrices.

Sur le V100, chaque Tensor Core traite des matrices 4×4 : 64 multiplications-additions par cycle. Les entrées sont en FP16, et l'accumulation peut se faire en FP32 pour garder la

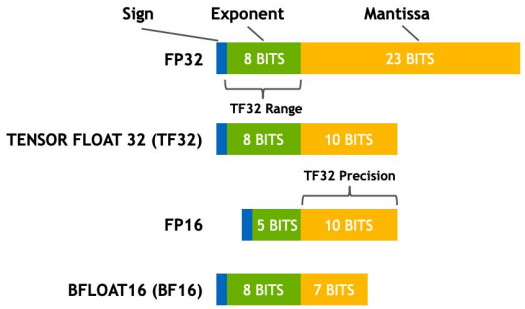
précision du résultat. Les générations suivantes traitent des blocs plus grands : 256 multiplications-additions par cycle pour un Tensor Core d'A100.

Comme l'apprentissage profond est fait à 80-90 % de multiplications de matrices et de convolutions, les Tensor Cores fournissent l'essentiel de la puissance du GPU. Mais à deux

conditions : utiliser une précision qu'ils acceptent, et des dimensions adaptées. Sinon, le calcul retombe sur les cœurs CUDA classiques, beaucoup plus lents.

Precisions & Tensor Cores

	NVIDIA H100	NVIDIA A100	NVIDIA Volta
Supported Tensor Core Precisions	**FP8**, FP64, TF32, bfloat16, FP16, ...	FP64, TF32, bfloat16, FP16, INT8, INT4, INT1	FP16
Supported CUDA® Core Precisions	FP64, FP32, FP16, bfloat16, INT8	FP64, FP32, FP16, bfloat16, INT8	FP64, FP32, FP16, INT8



Source : NVidia

Each Tensor Core generation supports certain precisions.

Each floating-point precision uses a fixed number of bits: one sign bit, an exponent and a mantissa.

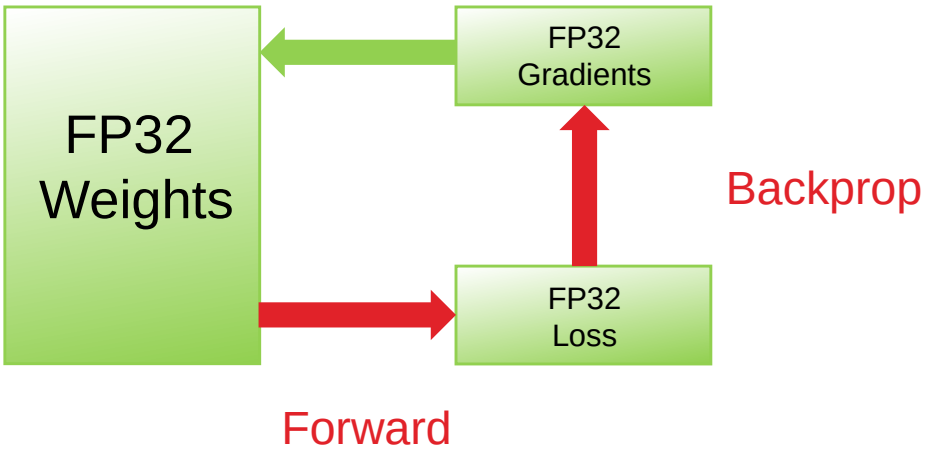
On the V100, only FP16 can be accelerated by the Tensor Cores.

Les différentes versions des *Tensor Core* utilisent certaines précisions.

Chaque précision de chiffre flottant utilise un nombre de bit fixé avec un bit de signe, un exposant et une mantisse.

Pour les V100, seul la précision FP16 peut être accélérer par les *Tensor Core*.

Normal Precision Training Loop



The starting point: a training loop entirely in FP32. The weights are FP32; the forward pass produces an FP32 loss; backpropagation computes FP32 gradients; the optimizer updates the weights.

Everything goes through the classic CUDA cores, except convolutions, which use TF32 on A100. It is simple and numerically safe, but we leave most of the Tensor Core power unused, and each value takes 4 bytes in memory.

The next slides show how to move some parts to reduced precision without losing training quality.

Le point de départ : une boucle d'entraînement entièrement en FP32. Les poids sont en FP32 ; le forward produit une loss en FP32 ; la rétropropagation calcule des gradients en

FP32 , l'optimiseur met à jour les poids.

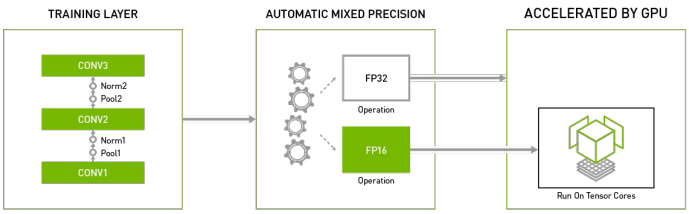
Tout passe par les cœurs CUDA classiques, sauf les convolutions qui, sur A100, utilisent le TF32. C'est simple et numériquement sûr, mais on laisse de côté la majeure partie de la

puissance des Tensor Cores, et chaque valeur occupe 4 octets en mémoire.

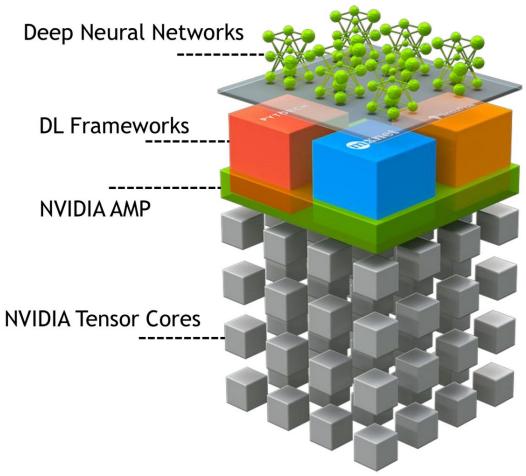
Les slides suivantes montrent comment passer certaines parties en précision réduite sans perdre en qualité d'apprentissage.

Automatic Mixed Precision

- Automatic Mixed Precision :
  - Necessary with V100 to use Tensor Core
  - The A100s use Tensor Cores with or without MP



- Pros:
  - **Insignificant** loss of precision for model training (gradient, loss, accuracy)
  - **Reduces memory** footprint
  - **Speeds up** calculations
- 2 steps to code :
  - Transforms eligible layers into FP16
  - Uses scaling to calculate gradients



17

Mixed precision means doing the heavy computations in reduced precision, FP16 or BF16, and keeping the sensitive parts in FP32.

AMP automates this choice operation by operation: convolutions and matrix multiplications move to reduced precision, on the Tensor Cores; sensitive operations such as reductions, softmax, normalizations and losses stay in FP32.

On V100, it is required to use the Tensor Cores. On A100 and H100, TF32 already uses them, but FP16 or BF16 are still twice as fast.

The benefits: faster computation, less memory (activations take half the space), and a negligible loss of precision in the vast majority of cases.

Mixed precision means switching some or all parameters to half precision (FP16) to use the Tensor Cores. It is required for the V100 Tensor Cores, but it needs a little code.

At the cost of a loss of precision that is almost always negligible in the gradients, the loss or the accuracy, it reduces the memory footprint and speeds up computation.

Automatic Mixed Precision for deep learning automates the mixed-precision principle. It converts the layers that are eligible (for example conv layers are, BatchNorm layers are not) and uses a scaling system for the gradient computation (see the next slide).

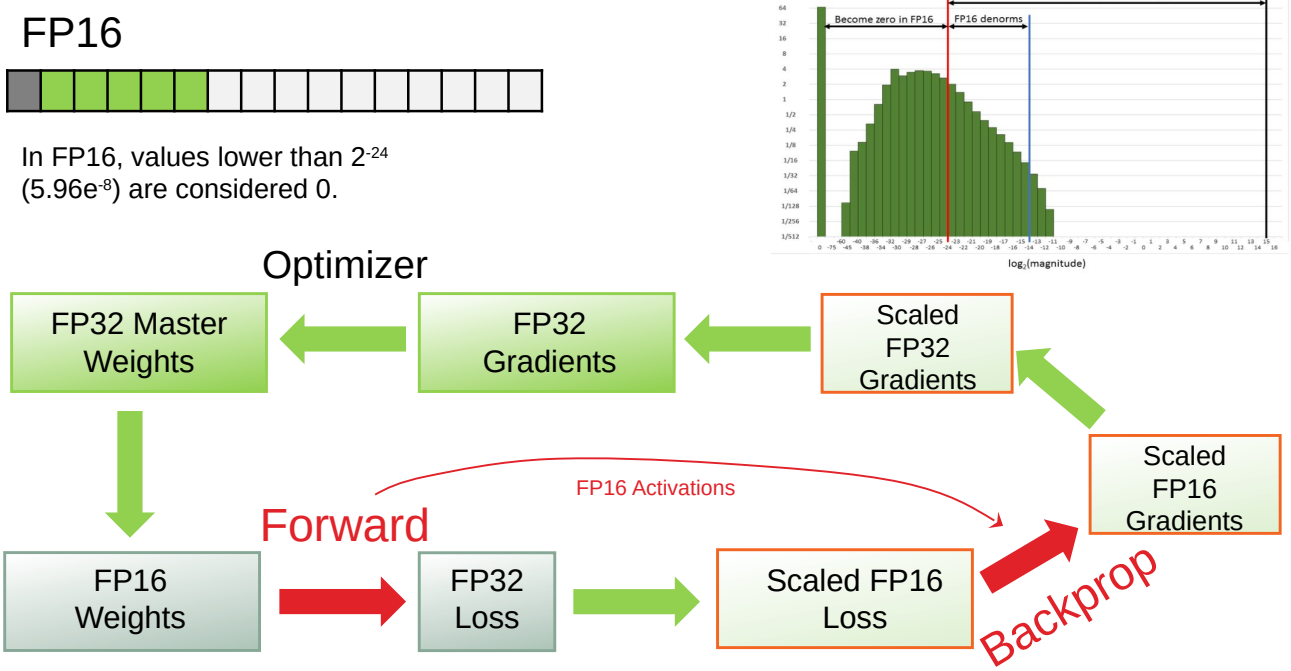
Also note: in PyTorch, matrix multiplications do not use TF32 by default; and "Uses scaling" applies only to FP16, not to BF16.

La précision mixte consiste à faire les calculs lourds en précision réduite, FP16 ou BF16, et à garder en FP32 ce qui est sensible.

L'AMP automatise ce choix opération par opération : les convolutions et multiplications de matrices passent en précision réduite, sur les Tensor Cores ; les opérations sensibles comme les réductions, softmax, normalisations et pertes restent en FP32.

Sur V100, c'est indispensable pour utiliser les Tensor Cores. Sur A100 et H100, le TF32 les utilise déjà, mais le FP16 ou le BF16 restent deux fois plus rapides.

AMP FP16 with Scaler



The scaler is often needed with AMP because computed gradients can reach very small values, very close to zero. FP16 treats every value below 5.96e-8 as exactly zero. Also, very large gradient values do not really occur in practice.

The problem with FP16: its range is narrow. Any value below about 6·10■■ becomes zero. As the histogram shows, part of the gradients is smaller than that. These gradients would vanish and the corresponding weights would stop learning.

The solution is scaling: before backpropagation, the loss is multiplied by a large factor, for example 65,536. By the chain rule, all gradients are multiplied by the same factor and move back into the FP16 range. Before the update, the gradients are divided by this factor, in FP32, and the optimizer updates the master weights, which are kept in FP32.

The factor is dynamic: the GradScaler increases it regularly, and if it detects infinite values, it skips the iteration and reduces it.

In practice, AMP converts the model's FP32 weights to FP16 (for the eligible weights), then the forward pass runs on the Tensor Cores.

Le principe du *Scaler* pour l'AMP est souvent nécessaire car les gradients calculés peuvent atteindre des valeurs très petites ou très proches

du zéro. En effet, la précision FP16 considère toutes valeurs inférieures à 5.96e-8 comme une valeur nulle (0 absolu). De plus les valeurs de

gradient très élevées n'existent pas en réalité.

Le problème du FP16 : sa plage est étroite. Toute valeur inférieure à environ 6.10■■ devient zéro. Or, comme le montre l'histogramme, une partie des gradients est plus petite que ça.

Ces gradients s'annuleraient et les poids correspondants n'apprendraient plus.

La solution est le scaling : avant la rétropropagation, on multiplie la loss par un grand facteur, par exemple 65 536. Par la règle de dérivation, tous les gradients sont multipliés par le

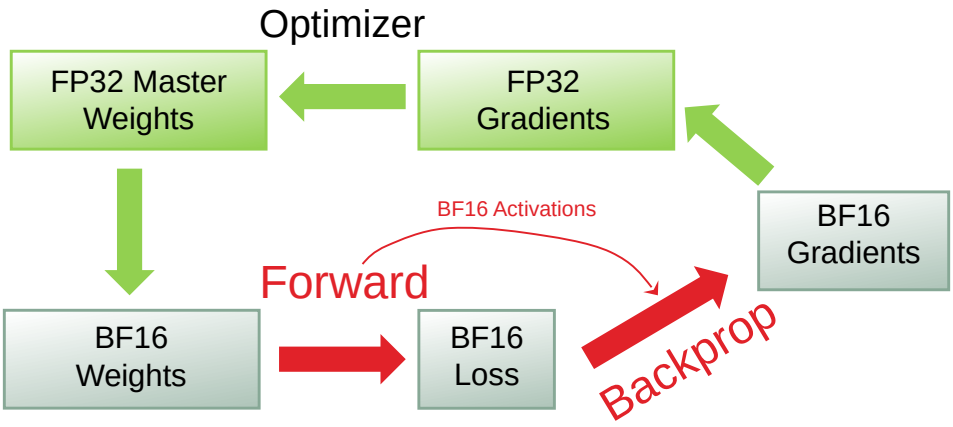
même facteur et remontent dans la plage du FP16. Avant la mise à jour, on divise les gradients par ce facteur, en FP32, et l'optimiseur met à jour les poids maîtres, conservés en

FP32.

AMP BF16 without Scaler

BF16





With BF16, the problem disappears. Its exponent has 8 bits, like FP32: it covers the same range of values, and small gradients do not become zero. No scaler needed.

We lose precision, with 7 mantissa bits instead of 10 for FP16, but deep learning is not very sensitive to it. And the master weights stay in FP32 for the update.

Limitation: BF16 is accelerated by the Tensor Cores only from the A100 onwards. On V100, you must stay in FP16 with a scaler. On A100 and H100, BF16 is now the default choice.

Avec le BF16, le problème disparaît. Son exposant fait 8 bits, comme le FP32 : il couvre la même plage de valeurs, et les petits gradients ne deviennent pas nuls. Plus besoin de

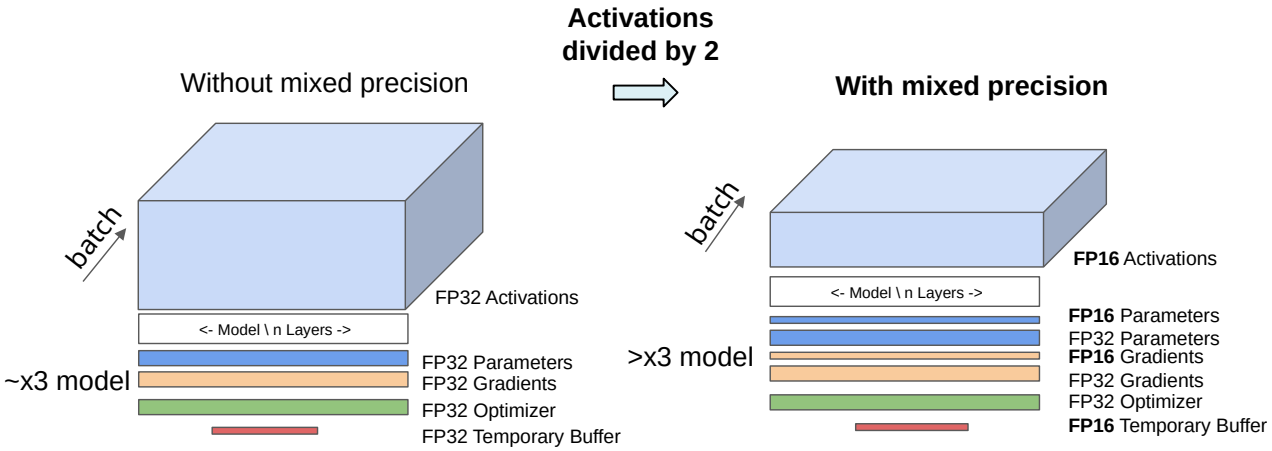
scaler.

On perd en précision, avec 7 bits de mantisse au lieu de 10 pour le FP16, mais l'apprentissage profond y est très peu sensible. Et les poids maîtres restent en FP32 pour la mise à

jour.

Limite : le BF16 n'est accéléré par les Tensor Cores qu'à partir de l'A100. Sur V100, il faut rester en FP16 avec scaler. Sur A100 et H100, le BF16 est aujourd'hui le choix par défaut.

Memory Footprint with Mixed Precision



Effect of mixed precision on memory. The big block represents the activations, kept during the forward pass for backpropagation. They grow with the batch size and dominate memory for a CNN. In mixed precision, they are FP16: their size is halved.

The model part, at the bottom, contains the parameters, the gradients and the optimizer states: about 3 times the model size with SGD with momentum, 4 times with Adam. Mixed precision increases it slightly, because of the FP16 copies of the weights. With torch.autocast, these copies are temporary.

Result: for most models, we save a lot of memory, which allows doubling the batch size. This is what the memory test of lab 2 will show. For very large models, the model part becomes dominant and other techniques are needed.

Effet de la précision mixte sur la mémoire. Le gros bloc représente les activations, conservées pendant le forward pour la rétropropagation. Elles grossissent avec la taille de batch et

dominent la mémoire pour un CNN. En précision mixte, elles sont en FP16 : leur taille est divisée par deux.

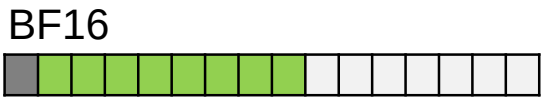
La partie modèle, en bas, contient les paramètres, les gradients et les états de l'optimiseur : environ 3 fois la taille du modèle avec SGD momentum, 4 fois avec Adam. La précision

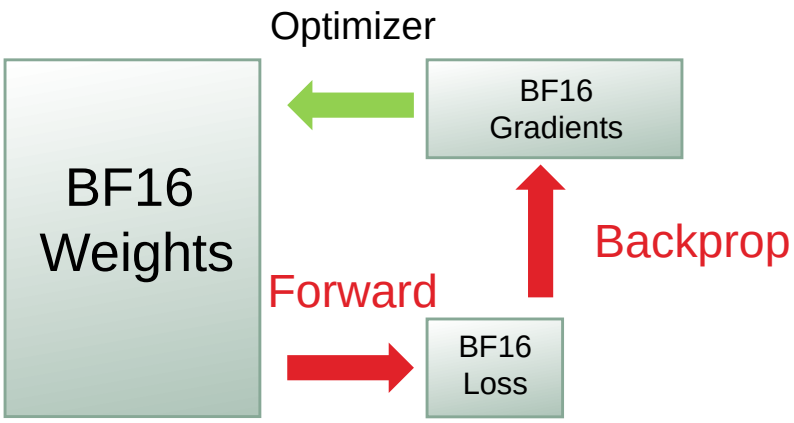
mixte l'augmente légèrement, à cause des copies FP16 des poids. Avec torch.autocast, ces copies sont temporaires.

Bilan : pour la plupart des modèles, on gagne beaucoup de mémoire, ce qui permet de doubler la taille de batch. C'est ce que montrera le test mémoire du TP2. Pour les très gros

modèles, la partie modèle devient dominante et il faut d'autres techniques.

Full Degraded Precision (BF16)





One step further: put everything in BF16, including the weights, the gradients and the optimizer states. No more FP32 copy: the model memory is halved, on top of the activation memory.

The risk: with only 7 mantissa bits, a small update added to a large weight can be lost to rounding. If the update is more than 256 times smaller than the weight, the weight does not change. Over a long training run, learning can stall.

In practice, this mode is combined with safeguards: stochastic rounding, rounding-error compensation, or optimizer states kept in FP32. Many large-model trainings actually keep FP32 master weights and only compute in BF16. So this mode is an option to save memory, not a default setting.

Une étape plus loin : tout passer en BF16, y compris les poids, les gradients et les états de l'optimiseur. Plus de copie FP32 : la mémoire du modèle est divisée par deux, en plus de

celle des activations.

Le risque : avec seulement 7 bits de mantisse, une petite mise à jour ajoutée à un grand poids peut être perdue par arrondi. Si la mise à jour est plus de 256 fois plus petite que le

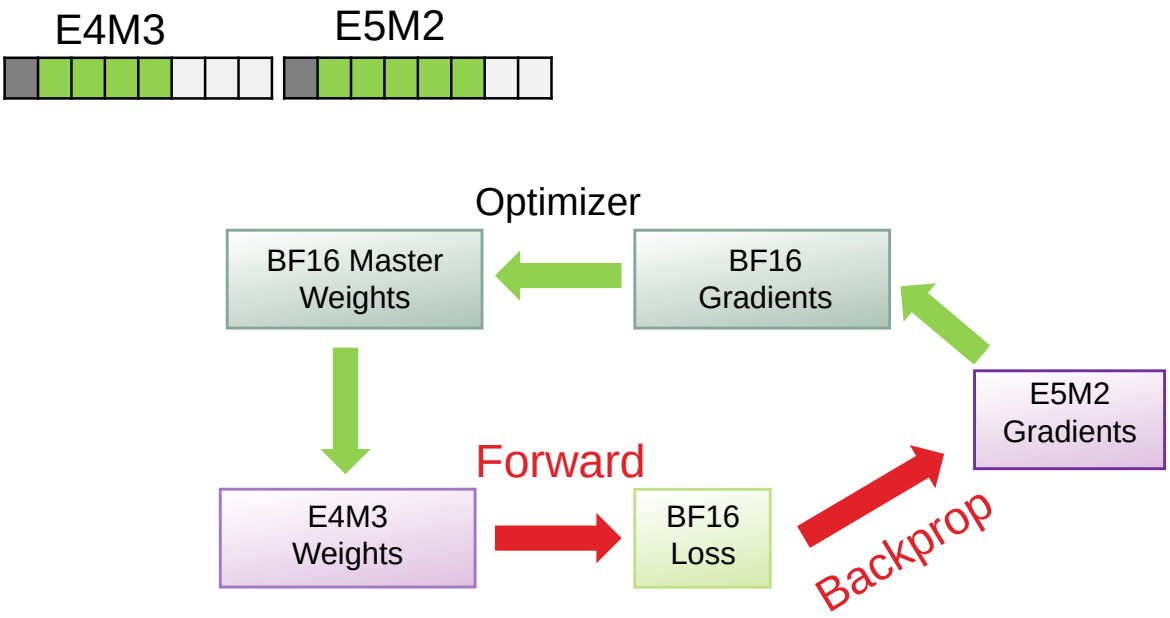
poids, le poids ne bouge pas. Sur un long entraînement, l'apprentissage peut stagner.

En pratique, on combine ce mode avec des précautions : arrondi stochastique, compensation des erreurs d'arrondi, ou états de l'optimiseur gardés en FP32. Beaucoup

d'entraînements de gros modèles conservent d'ailleurs des poids maîtres en FP32 et ne font que les calculs en BF16. Ce mode est donc une option pour gagner de la mémoire, pas

un réglage par défaut.

FP8 Mixed Precision



FP8, available on H100. Two 8-bit formats: E4M3 (4 exponent bits, 3 mantissa bits), more precise, used in the forward pass for weights and activations; and E5M2 (5 exponent bits, 2 mantissa bits), with a wider range, used in the backward pass for gradients.

Only the inputs of matrix multiplications move to FP8; accumulation is done in higher precision, and the loss, normalizations and optimizer stay in BF16 or FP32. Because the FP8 range is very narrow, each tensor has its own scaling factor. It is the same principle as the FP16 scaler, applied tensor by tensor.

The gain: about twice the BF16 throughput on the H100 Tensor Cores. NVIDIA's Transformer Engine library handles all of this. It is meant for very large models; for our lab, BF16 is enough.

Le FP8, disponible sur H100. Deux formats sur 8 bits : E4M3 (4 bits d'exposant, 3 de mantisse), plus précis, utilisé au forward pour les poids et les

activations ; et E5M2 (5 bits d'exposant, 2 de mantisse), plus large en plage, utilisé au backward pour les gradients.

Seules les entrées des multiplications de matrices passent en FP8 ; l'accumulation se fait en précision supérieure, et la loss, les normalisations et l'optimiseur restent en BF16 ou

FP32. Comme la plage du FP8 est très étroite, chaque tenseur a son propre facteur d'échelle. C'est le même principe que le scaler du FP16, appliqué tenseur par tenseur.

Le gain : environ deux fois la puissance du BF16 sur les Tensor Cores du H100. La bibliothèque Transformer Engine de NVIDIA gère tout cela. C'est réservé aux très gros modèles ;

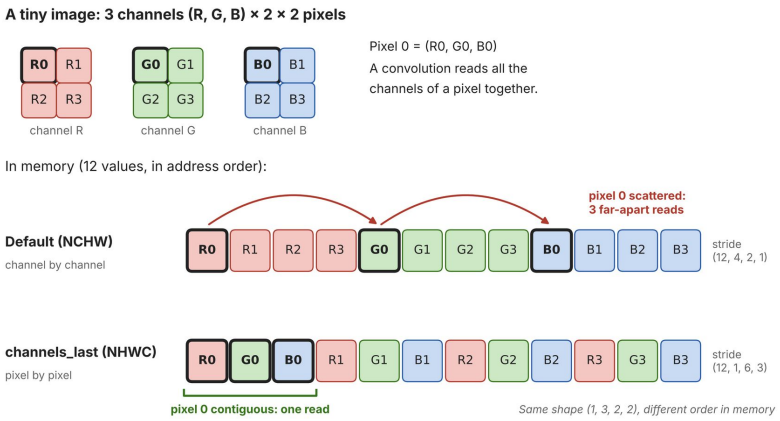
pour notre TP, le BF16 suffit.

Channel last memory format

batch channel height width

NCHW ← memory contiguity by default

.shape()



Take a tiny image: 3 channels, red, green, blue, of 2 × 2 pixels. That is 12 values. The tensor shape is (1, 3, 2, 2), but the shape does not say in which order these 12 values are stored in memory.

By default (NCHW), PyTorch stores channel by channel: the 4 red values, then the 4 green ones, then the 4 blue ones. But a convolution needs, for each pixel, all its channel values at the same time. For pixel 0, it must fetch R0, G0 and B0 from three distant places in memory.

In channels last (NHWC), values are stored pixel by pixel: R0, G0, B0 are side by side and read in one go. The shape does not change, only the order in memory changes: this is what the stride describes.

Prenons une image minuscule : 3 canaux, rouge, vert, bleu, de 2 × 2 pixels. Soit 12 valeurs. La forme du tenseur est (1, 3, 2, 2), mais la forme ne dit pas

dans quel ordre ces 12 valeurs sont rangées en mémoire.

Par défaut (NCHW), PyTorch range canal par canal : les 4 valeurs rouges, puis les 4 vertes, puis les 4 bleues. Or une convolution a besoin, pour chaque

pixel, de toutes ses valeurs de canaux en même temps. Pour le pixel 0, il faut aller chercher R0, G0 et B0 à trois endroits éloignés de la mémoire.

En channels last (NHWC), on range pixel par pixel : R0, G0, B0 sont côte à côte, lus en une fois. La forme ne change pas, seul l’ordre en mémoire

change : c’est ce que décrit le stride.

TP2&3 : Automatic Mixed Precision



- Activer l’Automatic Mixed Precision
- Test Mémoire
- Activer le channel last memory format

